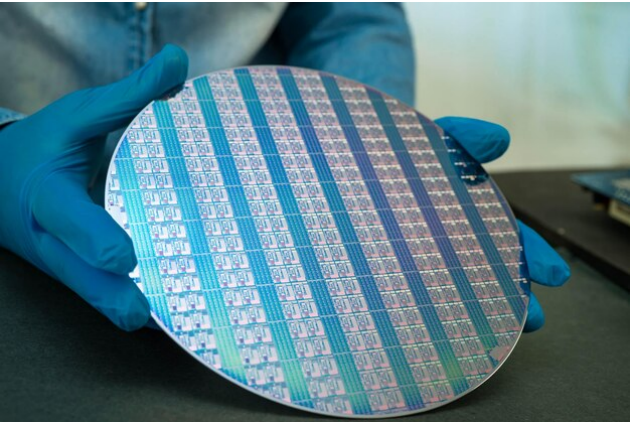
Tiny Tapeout Beginner Session (Digital Design)

Welcome!



About Me

Jack Thoene

ECE PhD Student · VAK Embodied System Lab

- Ultra-low-power systems, battery-free electronics
- A systems problem, not just a chip problem
- Why a systems person needs to understand silicon

What You Can Learn at Northwestern

- **CE 303** — Digital logic from scratch (the basics)
- **CE 355** — Write HDL, run it on real FPGAs
- **CE 357** — How the EDA tools actually work
- **CE 361** — Build a CPU
- **CE 387** — Synthesis + UVM verification
- **CE 391** — Transistors $\rightarrow$ standard cells $\rightarrow$ SRAM
- **CE 392** — VLSI project (simulated only)
- **CE 393** — Same idea, harder
- **ME 282** — Soft-litho microfluidics ($\sim$1 μm features)

Everything except one thing:

taping out.

Tonight we fix that.

What This Lab Is / Is Not

This Lab IS:

- A hands-on taste of the open-source chip design flow
- Guided — every step has instructions
- Beginner-friendly, no prerequisites

This Lab IS NOT:

- A full course in VLSI
- Something you need to finish today
- Graded

Open Source Is Coming for Chip Design

KiCad

2004	University toy
2013	CERN adopts it
2018	Adafruit, Arduino
2024	SpaceX, Google, Meta

Chips

2018	DARPA $\rightarrow$ OpenROAD
2020	Sky130, OpenLane
2022	Tiny Tapeout
2026	**You**

We are where KiCad was in 2013.

But the Commercial Stack Still Exists

	Cadence	Synopsys
Synthesis	Genus	Synplify / Design Compiler
Place & route	Innovus	ICC2
Simulation	Xcelium	VCS
Layout	Virtuoso	Custom Compiler
Signoff STA	Tempus	PrimeTime

Excellent tools. NU has the licenses. $1M–$5M/year for a real team. We are not using them today — because the open stack is the only one where *you* keep the keys.

"Synopsys offered Intel an all-you-can-eat EDA license exceeding $1 billion."

— Wing Venture Capital, "How Synopsys and Cadence are fueling the semiconductor industry" (*Intel has ∼10 K design engineers*)

And You Can Still Take It Anywhere

The toolchain is open. The PDK is open. **Your design is yours.**

- **Tonight:** prototype on TT for $150 (Apache 2.0, public)
- **Tomorrow:** same Verilog → same Yosys / OpenLane → commercial MPW
- **Real shuttles that take open-tool flows:** Efabless, Muse Semiconductor (TSMC), GlobalFoundries 22FDX
- **Or go fully closed** with Cadence/Synopsys — the RTL is portable

TT defangs the IP problem by making the IP *irrelevant* — there is nothing for tech transfer to claim, because it is already public.

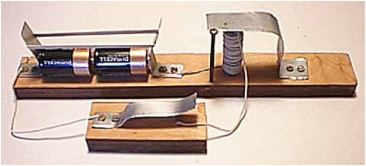
Today's Plan

- How we got here
- A transistor: what it is, how it switches
- Build one in SiliWiz
- Combine them into logic gates
- Simulate them in Wokwi
- **Build a real chip in 130 nm CMOS** (90 minutes)
- Where HDLs fit
- Run the whole pipeline locally

No matter how small the functionality: this is a real chip. That is the point.

What Is Digital Communication?

- Information as **discrete symbols**
- 1840s telegraph: dots and dashes
- Baudot, ASCII, binary streams
- Same insight: **two states**

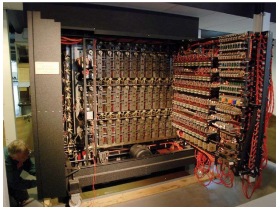


Takeaway: Morse → Baudot → ASCII → binary → the bits in your laptop right now.

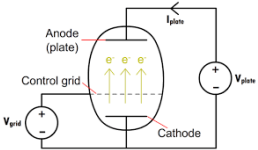
Image: Google Images

From Logic to Computation

- **1930s** Turing → binary tape
- Boolean logic: AND, OR, NOT
- **1940s** vacuum tubes as switches
- ENIAC: 17,468 of them, one room
- Big, hot, expensive, unreliable



Takeaway: A vacuum tube is just a switch. So is a transistor. Same function, a billion times smaller.



Images: MathWorks, Google Images

The Transistor Revolution

- **1947** transistor → smaller, faster, cheaper
- **1960s** multiple transistors on one chip = IC
- **1970s** thousands. **2020s** tens of billions.
- VLSI: shrinking features, growing complexity

But as complexity grew, so did secrecy.

How a Transistor Works

Voltage-controlled switch.

A voltage on the **gate** induces a field that lets current flow from **source** to **drain**.

- Gate low → **OFF**
- Gate high → **ON**

The physics is CE 391. Today we just need: current flows source → drain, on/off by the gate.

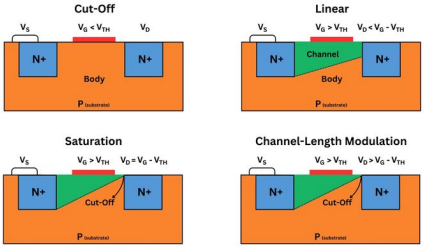


Image: Wevolver, "How an NMOS Transistor Works"

How Is a Transistor Actually Made?

A transistor is built *layer by layer* on a silicon wafer using a process called **photolithography**:

1. **Oxidation** — grow a thin insulating SiO_2 layer
2. **Deposition** — coat with photoresist
3. **Exposure** — shine UV light through a mask
4. **Develop** — wash away resist to reveal a pattern
5. **Etch / Implant** — remove material or implant dopants
6. **Repeat** — stack layers: diffusion, poly, metal

Each layer is aligned to sub-nanometre precision. A modern chip may require 80+ mask steps. The sky130 process we use tonight has ∼40.

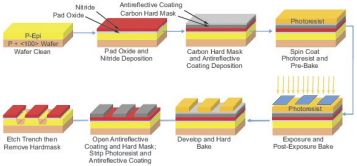


Image: Newport Corporation

Why Does This Matter for Us?

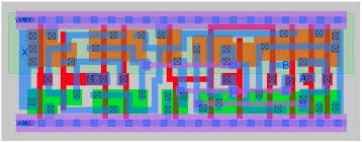
Full physics: take EE 391.

- Transistors = patterned layers
- Layers have **design rules**
- Rules live in the **PDK**
- SiliWiz draws those same layers

You don't have to run a fab. You have to know what the fab expects — the tools enforce it.

The Secret Sauce: PDKs

- **PDK** = the foundry's spec
- Locked behind NDAs + six-figure fees
- No PDK = no chip
- **2020:** Google + SkyWater open-sourced **sky130**



Tonight we build a chip using that open PDK. No NDA. No license fee.

Image: Google Images

From Open PDK to Open Fabrication

- Open PDK → *design* solved
- Traditional 130 nm shuttle: $30K–$50K
- Full mask set: $500K+
- **Efabless** → shared shuttles
- **2020** Google-sponsored free MPWs
- But: months of setup, EDA expertise

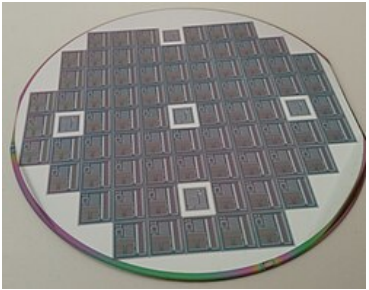


Image: Wikimedia Commons

Tiny Tapeout: Chip Design for Everyone

Tiny Tapeout (Matt Venn) $\rightarrow$ radically accessible:

- A **tile**: $\sim$160$\times$100 μm, $\sim$1,000 gates
- Hundreds of tiles per chip, costs split
- Tile + board + chip: $\sim$**\$100–\$300**
- Open tools, GitHub templates, CI does the hardening.
- Timeline: $\sim$6–9 months from submission to chip in hand

Traditional fab: \$500K. Efabless shuttle: \$0 but hard. Tiny Tapeout: \$150 and you can do it tonight.

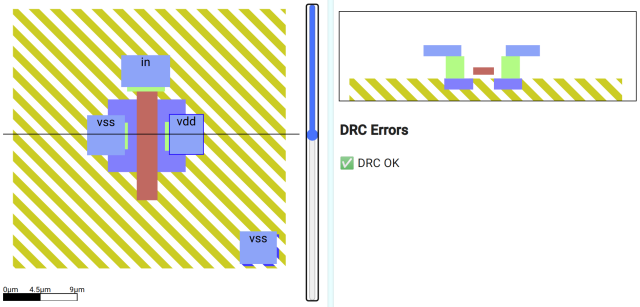
What You Need

- Laptop + charger
- GitHub account (github.com)
- Chrome or Firefox

Takeaway: No installs. No setup. Everything runs in the browser for Levels 1–3.

Level 1: Draw a Transistor in SiliWiz

1. Open SiliWiz. Select n-diffusion, draw a horizontal rectangle.
2. Select polysilicon. Draw a vertical rectangle crossing the diffusion.
3. Select metal. Place contacts at source, drain, and gate.
4. Switch to Cross-section view.
5. Switch to Simulation view — sweep gate voltage.



Takeaway: Like learning what a resistor is before plugging it into a breadboard. You now know what is inside every transistor symbol you will see today.

Level 1: What You Just Built

The rectangle you drew is a real **NMOS transistor** laid out the same way the fab would build it.

- **n-diffusion** = source and drain (doped silicon)
- **poly** crossing it = the gate
- **metal contacts** = how the outside world reaches each terminal

When you swept the gate voltage in Simulation view, you watched the channel form. Below threshold: no current. Above threshold: current flows from source to drain. **That is the switch.**

Every transistor in every chip on this planet works this way. You just built one.

How a NAND Gate Is Built on Silicon

A CMOS NAND gate is just 4 transistors arranged in a specific topology:

- **2 NMOS in series** (pull-down) — both must be ON to pull output LOW
- **2 PMOS in parallel** (pull-up) — either ON pulls output HIGH

Every standard cell in a PDK is one of these arrangements, hand-optimized and characterized.

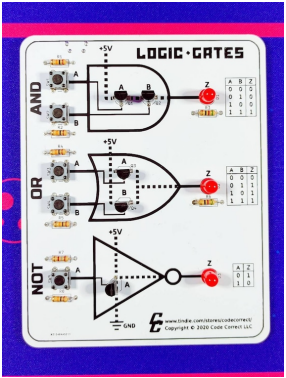


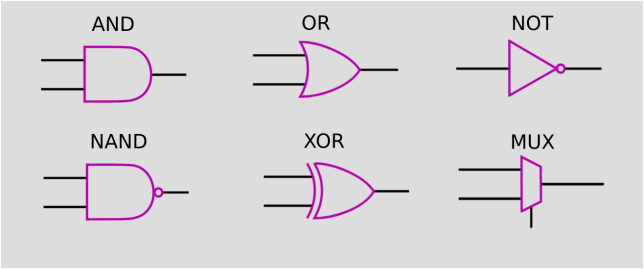
Image: Etsy

Takeaway: Full cell layout is an EE 391 topic. We are about to build them in Wokwi.

Level 2: NAND Gate — Truth Table

1. Toggle inputs 1 and 2 with the DIP switches
2. Find the combination that turns the red LED on
3. Verify: NAND is LOW only when *both* inputs are HIGH

Takeaway: This is two transistors wired together. Every gate in every chip works this way.



Level 2: The Flip-Flop — Adding Memory

1. This is a D flip-flop — it *remembers* a value
2. When Enable = 1: output follows the data input
3. When Enable = 0: output holds its last value

Combinational logic (NAND) has no memory. **Sequential** logic (flip-flop) remembers state. Together they can compute anything.

Takeaway: A flip-flop is just a few gates with feedback. This is how your laptop stores a bit.

Level 2: The Full Adder — How Fast It Scales

1. Toggle A, B, and Carry-in with the switches
2. Observe Sum (cyan LED) and Carry-out (orange LED)
3. Adds two 1-bit numbers: $A + B + C_{in} = S, C_{out}$

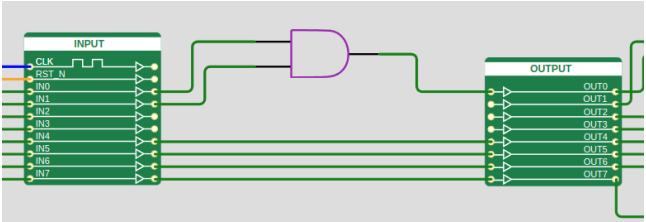
5 gates to add two bits. Chain 32 of these = a 32-bit adder. Chain 64 = the ALU in your laptop.

You can see why drag-and-drop does not scale.

Takeaway: This is why we need Verilog. Describe the logic in text, let the compiler wire the gates.

Level 2: Build Your Own Gate in Wokwi

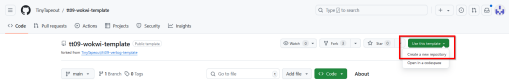
1. Open the blank Wokwi template. Save immediately (Ctrl+S).
2. Press +, add a 74HC00 (Quad NAND).
3. Wire: IN0 → 1A, IN1 → 1B, 1Y → OUT0.
4. Power: VCC → pin 14, GND → pin 7.
5. Click Play. Toggle inputs. Verify NAND truth table.



Takeaway: This is your breadboard. If it works here, it works on silicon.

Level 3: Fork the Template

1. Go to `github.com/TinyTapeout/ttsky-wokwi-template`
2. Click **Use this template** → Create a new repository
3. Set visibility to **Public**



Takeaway: This repo is your Arduino project folder. Everything from here feeds into the compiler.

Level 3: Link Your Wokwi Design

1. Open `info.yaml` in your repo
2. Under `project:`, set `wokwi_id` to the number in your Wokwi URL
3. Fill in `title`, `author`, `description`. Commit.

TinyTapeoutTest / info.yaml

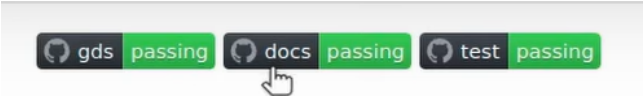
Code Blame 48 lines (42 loc) · 1.45 KB

1.45 KB

```yaml

# The pinout of your project. Leave unused pins blank. DO NOT delete or add any pins.
pinout:
  # Inputs
  ui[0]: "input a"
  ui[1]: "input b"
  ui[2]: ""
  ui[3]: ""
  ui[4]: ""
  ui[5]: ""
  ui[6]: ""
  ui[7]: ""

  # Outputs
  uo[0]: "output and"
  uo[1]: "output nand"
  uo[2]: "output or"
  uo[3]: "output clock div 16"
  uo[4]: "output clock div 12"
  uo[5]: ""
  uo[6]: ""
  uo[7]: ""

```

Level 3: Watch the Action

1. Go to the **Actions** tab. Watch `gds` run (yellow → green).
2. ~5 min: schematic → synthesis → place-and-route → DRC.
3. Wait for both `gds` and `docs` to show green.



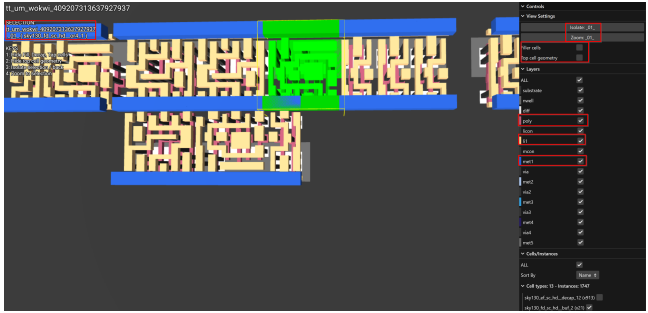
Takeaway: You just clicked Upload. The green checkmark means your design compiled, passed DRC, and is ready to fabricate.

Level 3: View in 3D

1. Settings → Pages → source: **GitHub Actions**
2. Visit your GitHub Pages URL. Toggle layers on and off.

The viewer shows the full tile, not just your logic. Most of what you see is routing, tap cells, and antenna diodes added by the toolchain. Your design is the small cluster of cells inside.

Takeaway: This is what the fab actually builds — like inspecting the compiled binary, but for hardware.



From Truth Table to Gates — Automatically

Remember the NAND truth table? You wired it by hand in Wokwi. But what if you had 4 inputs and 7 outputs — like a BCD-to-7-segment decoder?

The **Wokwi LUT Generator** does it for you:

1. Write a truth table in JSON (inputs → outputs)
2. The tool converts it to boolean equations (Quine-McCluskey)
3. It generates a complete Wokwi schematic — AND/OR gates wired

This is exactly what Verilog and Yosys do.
You describe *what* the logic should do. The tool figures out *how* to build it. Verilog is the same idea, but for circuits too complex for a truth table.

Takeaway: Try it: `tinytapeout.com/digital_design/wokwi_lut_generator/`

OK — Let's Talk Verilog

Everything up to this point has been drag-and-drop. Wokwi is a great starting point, but it caps out at about 20 gates.

Real chip designs — counters, state machines, VGA controllers, CPUs — have thousands of gates. You cannot drag-and-drop your way there. You need a hardware description language.

Verilog.

Same idea as going from the Arduino block editor to writing real C++. You describe what the hardware should do in text, and a compiler (Yosys) turns it into gates.

Takeaway: The next two levels introduce Verilog. Stretch goals — do them tonight or on your own.

Level 4: What Is This Code?

Open **vga-playground.com** and pick **Stripes**. Here is what each piece does:

Verilog	What it does
`module tt_um_vga_example(...)`	Top-level box. Inputs/outputs are the 26 TT tile pins.
`wire [9:0] pix_x, pix_y;`	Current pixel position (0–639 across, 0–479 down).
`reg [9:0] counter;`	10-bit register that counts up by 1 each frame (∼60 Hz).
`wire [9:0] moving_x = pix_x + counter;`	Shifts pixel position right each frame → stripes scroll left.
`assign R = {moving_x[5], pix_y[2]};`	Picks one bit → stripe every 32 pixels.
`always @(posedge vsync)`	Runs once per frame. Increments `counter`.

Level 4: Two Kinds of Verilog

Everything in Verilog is one of two things:

	Combinational	**Sequential**
Syntax	`assign` / `wire`	`always @(posedge clk)`
When	Every time inputs change	Once per clock edge
Hardware	Pure wires and gates	Flip-flops (memory)
Example	`assign out = a & b;`	`counter <= counter + 1;`
Analogy	A light switch	A latch that remembers

Things that simulate but will NOT synthesize:

- `#10` delays — no such thing in real gates
- Division `/` — use `» 1` instead of `/ 2`
- Large memories — a 640×480 framebuffer is too big for one tile

Level 4 Goal: Reverse the Stripes

Find this line:

```
wire [9:0] moving_x = pix_x +
counter;
```

Change + to −.

That is it. One character. The stripes now scroll the other direction.

Why? `+ counter` shifts the pixel coordinate right each frame, so the pattern appears to move left.
`− counter` shifts the other way.

Try next: change `moving_x[5]` to `[4]` (16 px stripes) or `[6]` (64 px). Change which colours are on/off.

Templates and Module Naming

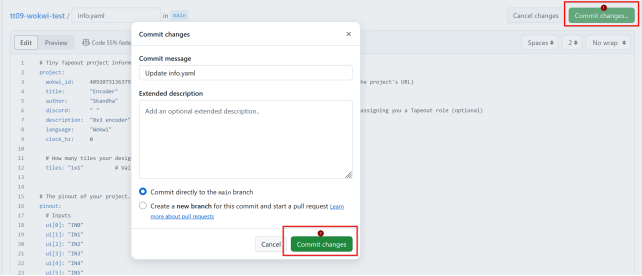
Pick a template: Wokwi, sky130 Verilog, IHP Verilog, analog

Top module must be `tt_um_YOURGITHUBNAME_projectname` — and `top_module` in `info.yaml` must match exactly.

Files you edit: `project.v`, `info.yaml`, `docs/info.md`, `test/test.py`

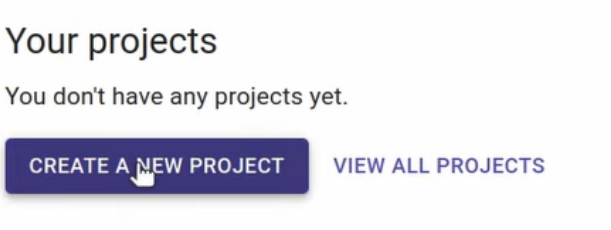
Level 4 Goal: GDS on GitHub Actions

1. Fork `TinyTapeout/ttihp-verilog-template`
2. Settings → Pages → source: GitHub Actions
3. Copy Verilog into `src/project.v`. **Rename module** (required).
4. For VGA: also copy `hvsync_generator.v` into `src/`
5. Update `info.yaml`: `top_module` + add `hvsync_generator.v` to `source_files`
6. Add `hvsync_generator.v` to `PROJECT_SOURCES` in `test/Makefile` (space-separated)
7. Update `docs/info.md`; add `cocotb.pass_test()` in `test/test.py`



Level 4: Submit

1. Go to `app.tinytapeout.com/projects/create`
2. Log in with GitHub
3. Create a new project, paste your repo URL
4. Apply coupon. Click **Submit a new revision**.



How Submission Actually Works

1. **Design passes CI** — both `gds` and `docs` green on GitHub Actions. Compiled, met timing, passed DRC. Fab-ready.
2. **Get a token** — distributed May 5. Each token reserves one tile. Your organizer hands them out.
3. **Submit at** `app.tinytapeout.com/projects/create` — log in with GitHub, paste your repo URL, enter token or pay ($∼150).
4. **Iterate until May 9** — update any time before the deadline. Commit changes, click "Submit a Revision".
5. **Wait ∼6–9 months** — your tile is fabricated on a shared wafer. You get a dev board with your chip.

Takeaway: You are good when: green CI + submitted on the app + confirmation email.

Your Tile's Interface: 26 Pins

Every TT tile has the same fixed interface. Your design plugs into it.

Signal	Width	Direction	Purpose
`clk`	1	in	Clock
`rst_n`	1	in	Active-low reset
`ena`	1	in	Enable (always high during operation)
`ui_in[7:0]`	8	in	Your inputs (switches, signals)
`uo_out[7:0]`	8	out	Your outputs (LEDs, signals)
`uio_in[7:0]`	8	in	Bidirectional input path
`uio_out[7:0]`	8	out	Bidirectional output path
`uio_oe[7:0]`	8	out	Bidir output enable (1 = output)

Rules:

- Unused outputs **must** be assigned `8'b0`
- If not using bidir pins: set `uio_out = 8'b0`, `uio_oe = 8'b0`
- Use `rst_n` (active-low) — no `initial` blocks

A Note on IP and Open Source

Tiny Tapeout requires your design to be **open-source (Apache 2.0)**. Your design and documentation will be **publicly published**. This means:

- You **keep ownership** of your design
- But anyone can use, modify, and build on it
- Publication counts as **public disclosure** for patent law
- **No confidentiality** — not compatible with NDAs

For students: almost always fine. You are building a learning project.

For research labs: talk to your tech transfer office *before* submitting.

Takeaway: Full terms: `tinytapeout.com/terms/`

Testing Your Design

Before you submit, you should test your design. TT uses **cocotb** — a Python framework that drives your Verilog inputs and checks the outputs.

1. Write a Python test in `test/test.py`
2. Set inputs, wait a clock cycle, assert outputs
3. Run locally: `make -C test` (needs OSS CAD Suite)
4. Or just push — GitHub Actions runs your tests automatically
5. View waveforms: open `tb.vcd` in GTKWave

At minimum: add `cocotb.pass_test()` to pass CI.

Tapeout Day (optional): a group hackathon to write tests, fix bugs, polish, and submit before the May 9 deadline.

Takeaway: More: `tinytapeout.com/hdl/testing/`

Tapeout Assignments — Let's Talk

We have a limited number of tiles for this workshop, and this round is **digital only**.

- Who is committing to a tapeout this cycle?
- What is your design idea — in one sentence?
- Solo, or pairing up?
- Who needs help picking a project?

Tokens go out **May 5**. Submission deadline **May 9**. If you are still deciding, that is fine — but we need to know by the end of today so we can hand out tokens accordingly.

Analog or mixed-signal designs: see the advanced session.

Level 5: Install OSS CAD Suite

Download from `github.com/YosysHQ/oss-cad-suite-build/releases`

OS	File	Install
Linux (x64)	`*-linux-x64-*.tgz`	`tar -xf *.tgz`
Linux (ARM)	`*-linux-arm64-*.tgz`	`tar -xf *.tgz`
Mac (Apple Silicon)	`*-darwin-arm64-*.tgz`	`tar -xf *.tgz`
Mac (Intel)	`*-darwin-x64-*.tgz`	`tar -xf *.tgz`
Windows	`*-windows-x64-*.exe`	Run installer

Activate: `source`

`oss-cad-suite/environment` (Linux/Mac) or `oss-cad-suite\environment.bat` (Windows). Verify with `yosys -version`.

Takeaway: You just installed gcc. Same compiler the cloud was running.

Level 5: What Is OSS CAD Suite?

Not Quartus. No GUI, no project wizard. A bag of CLI tools you chain together:

Tool	What it does	Quartus equivalent
`yosys`	RTL synthesis (Verilog → gates)	Analysis & Synthesis
`nextpnr`	Place-and-route	Fitter
`iverilog`	Verilog simulation	ModelSim
`gtkwave`	Waveform viewer (VCD)	SignalTap
`sby`	Formal verification	—

Languages: Verilog (native) · SystemVerilog (`yosys-slang`) · VHDL (`ghdl`).

Editing: Any text editor. VS Code with the *Lushay Code* extension gives syntax highlighting and lint.

Takeaway: Think `gcc` + `gdb` + `make`, not Visual Studio.

Level 5: FPGA → ASIC — What to Fix

Issue	FPGA	ASIC fix
Power-on state	Flops start at 0	Use explicit `rst_n`
Reset polarity	Active-high	TT uses active-low
`initial` blocks	Work	Ignored — remove
Division /	Synthesizes	Use » `1`
`#delay`	Sim only	Remove

Level 5: Simulate and Synthesize

Example files: `examples/project.v` (4-bit counter), `examples/tb.v` (testbench).

Activate: `source oss-cad-suite/environment`
Then cd into the workshop repo (not oss-cad-suite):

Simulate:

```
iverilog -o sim examples/tb.v examples/project.v
vvp sim
```

View waveforms: `gtkwave tb.vcd`

Synthesize + view schematic:

```
yosys -p "read_verilog examples/project.v;
   synth -top tt_um_yourname_counter; stat; show"
```

Takeaway: The `stat` output is your cell inventory. `show` opens a gate-level schematic.

Level 5: Get Into Tiny Tapeout

1. Fork `ttsky-verilog-template` or `ttihp-verilog-template`
2. Copy your `.v` file to `src/project.v`
3. Update `info.yaml` — `top_module` must match module name
4. Add `cocotb.pass_test()` in `test/test.py`
5. Commit → green → `app.tinytapeout.com` → Submit

Key Deadlines

Date	Milestone
April 17	Beginner workshop (today)
May 1	IEEE form submission
May 5	Token distribution
May 9	**Final submission deadline**